CS1312
Algorithms Design and Analysis

Divide and Conquer

Algorithm Design Techniques

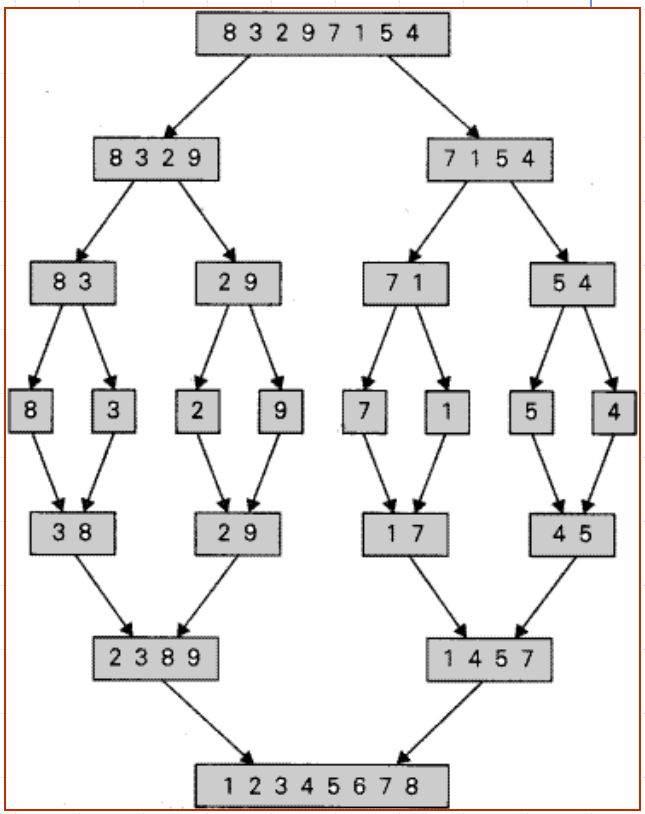
General approaches to solving problems algorithmically that is applicable to a variety of problems from different areas of computing.

- Brute force
- Divide and conquer
- Branch and bound
- Greedy approach
- Iterative improvement
- ……

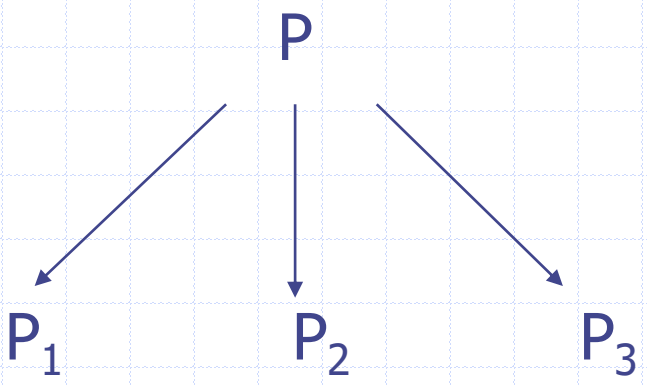
Divide-and-Conquer

- Divide-and-conquer paradigm solves a problem recursively, applying three steps at each level of the recursion:

1. **Divide** the problem into a number of subproblems that are smaller instances of the same problem.
2. **Conquer** the subproblems by solving them recursively.
3. **Combine** the solutions to the subproblems into the solution for the original problem.



Divide-and-Conquer (Cont.)



Problem

Subproblems

$P_1 + P_2 + P_3$ + overhead (splitting subproblems + combining the results)

Divide-and-Conquer (Cont.)

- **Recursive case :**

  When the subproblems are large enough to solve recursively

- **Base case:**

  Once the subproblems become sufficiently small, recursion is no longer applied.

Recurrences

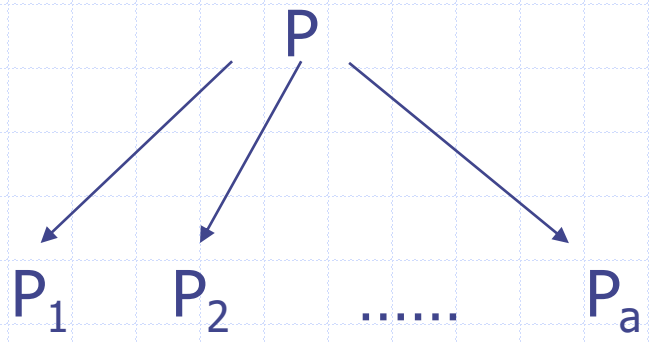
- When an algorithm contains a recursive call to itself, we can often describe its running time by a *recurrence equation* or *recurrence*.

- A **recurrence** is an equation or inequality that describes a function in terms of its value on smaller inputs.
  - i.e. describes the overall running time on a problem of size **n** in terms of the running time on smaller inputs (e.g, **½ n**).

T(n) = T(n/2)+c

```
Algorithm fun(n)
    if n>=1
        Print (n)
        fun (½n)
```

Recurrence for divide and conquer



Problem

Subproblems

Size of each $P_i = \frac{n}{b}$

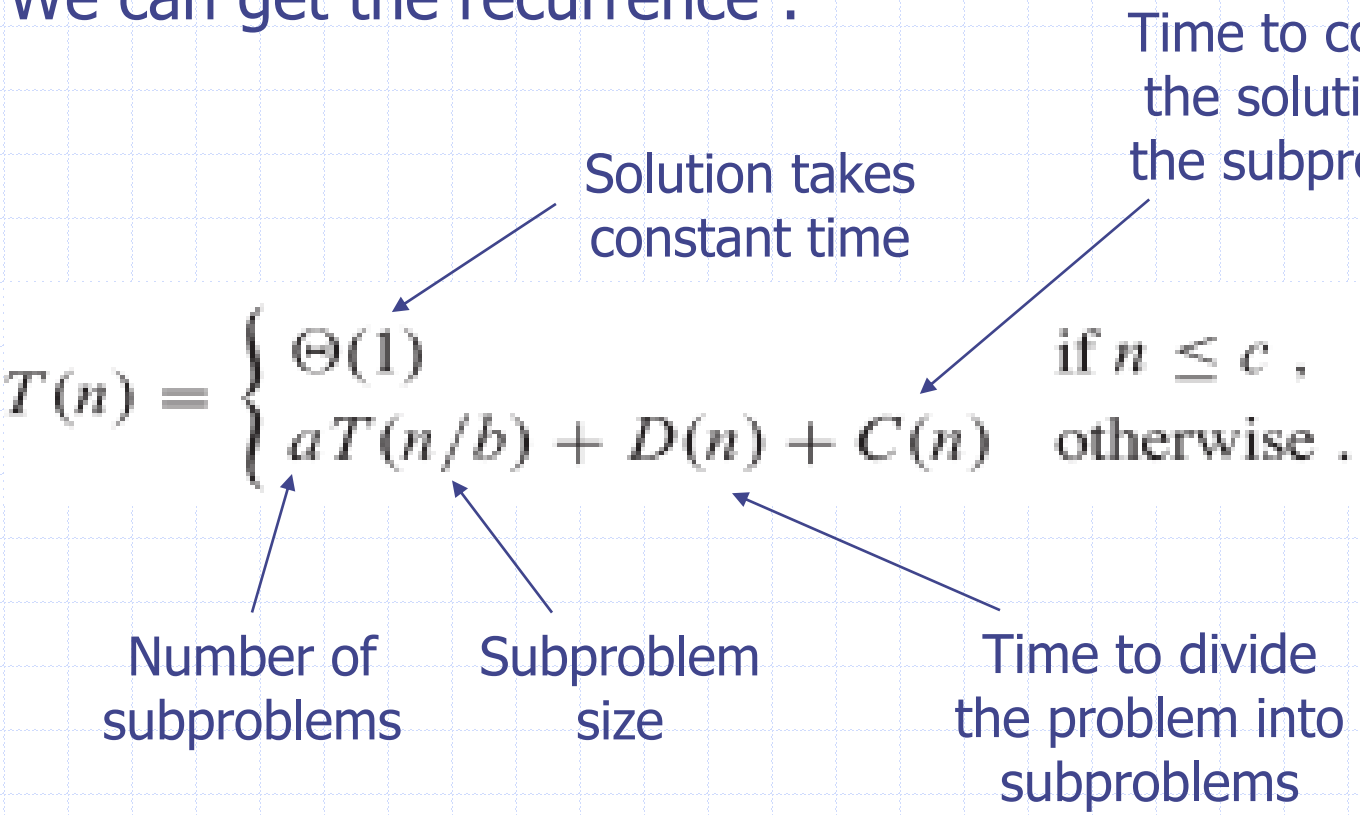
Recurrence for divide and conquer

Suppose that the problem division yields:

1. Let **a** subproblems, each of size **1/b** of the original. It takes **T(n/b)** time to solve **one** subproblem of size **n/b**. So it takes **aT (n/b)** time to solve **a** subproblems.
2. **D(n)** time to divide the problem into subproblems
3. **C(n)** time to combine the solutions of the subproblems into the solution of the original problem.

Recurrence for divide and conquer

We can get the recurrence :



Example

1. $T(n) = 2T(n/2) + f(n)$ [Merge Sort]
2. $T(n) = T(n/2) + f(n)$ [Binary Search]
3. $T(n) = T(n-1) + f(n)$ [Factorial]
4. $T(n)=T(n-1)+T(n-2)+f(n)$ [Fibonacci]
5. $T(n) = 2T(n/4) + T(2n/4) + f(n)$
   - Three branches, two of one-fourth and one of two-fourths.

Recurrence for divide and conquer

There are different methods for solving recurrences (obtaining θ or O):

- Iteration methods
- Recursion tree
- Master method
- Substitution method

The Iteration Methods

The iteration method

- In the iteration method we iteratively "unfold" the recurrence until we "see the pattern".
- The iteration method does not require making a good guess like the substitution method.

The iteration method

$$T(n) = \begin{cases} T(n-1) + 1 & \text{if } n>1 \\ 1 & \text{if } n=1 \end{cases}$$

```
Algorithm fun(n)
    if n>1
        Print (n)
        fun (n-1)
```

- $T(n) = T(n-1) + 1$

  $= T(n-2) + 1 + 1$

  $= T(n-3) + 1 + 1 + 1$

  $\vdots$ $\quad T(n-k) + k \quad \Rightarrow n-k = 0 \Rightarrow n = k$

  $= T(1) + \underbrace{1 + 1 + \ldots + 1}_{n-1}$

  $= \Theta(n)$

The iteration method

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1; \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

T(n) = n + 2T(n/2)

= n + 2(n/2 + 2T(n/4))

= n + n + 4T(n/4)

= n + n + 4(n/4 + 2T(n/8))

= n + n + n + 8T(n/8)

T(n) = 3n + 2^3T(n/2^3)

= kn + 2^kT(n/2^k)

= nlgn + nT(1)

T(n) = O(nlgn)

```
Algorithm fun(n)
    if n>1
        for (n)
            print()
        fun (n/2)
        fun (n/2)
```

After *k* steps

Algorithm stops when $n/2^k=1$

$n/2^K = 1 \Rightarrow n = 2^k$

$lgn = k$

Assume $n=2^k$ ➔ k = lg n

The Recursion Tree

The recursion-tree method

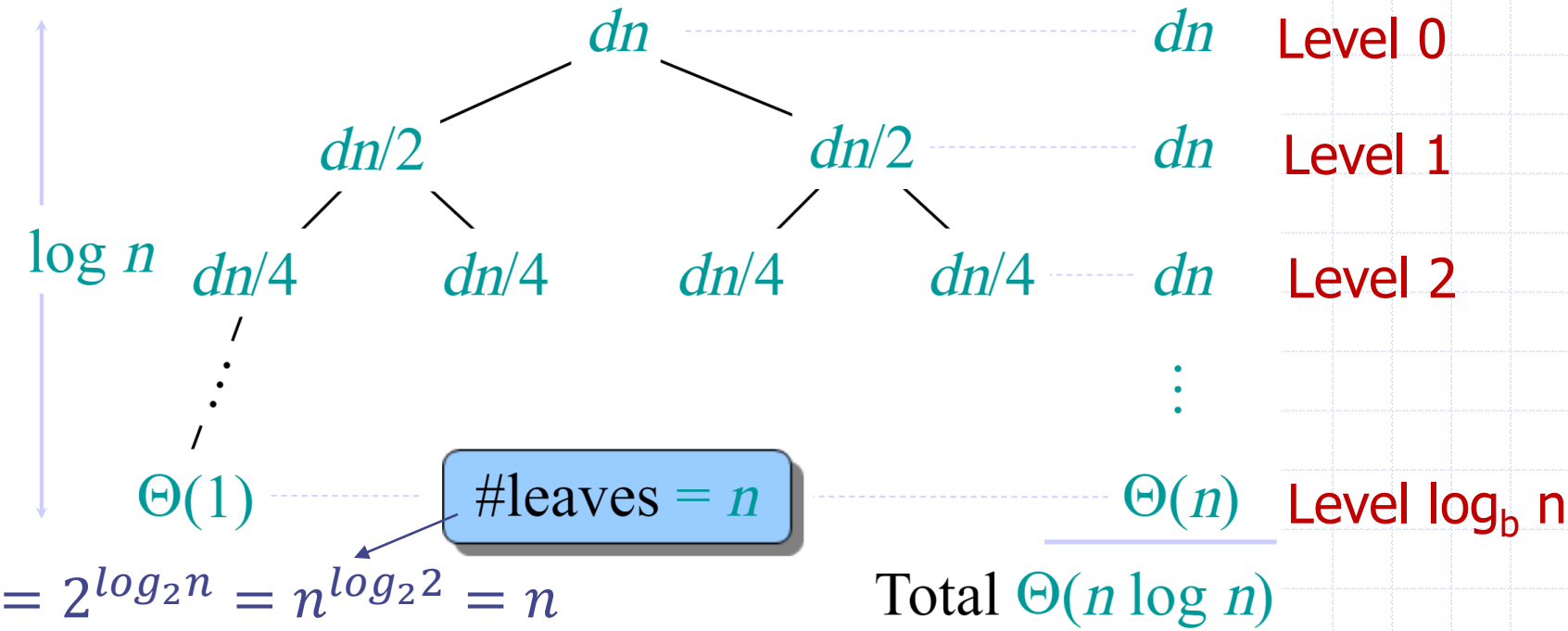
- Drawing out a recursion-tree is a straight forward way to devise a good guess.
- In a recursion tree, each node represents the cost of a single subproblem somewhere in the set of recursive function invocations.
- We sum the costs within each level of the tree to obtain a set of per-level costs.
- Then we sum all the per-level costs to determine the total cost of all levels of the recursion.

```
n = 8
Algorithm fun(n)
    if n>1
        for (n)
            print()
        fun (n/2)
        fun (n/2)
```

The recursion-tree method

- It is best used to generate a good guess, which is then verified by the substitution method.
- The expected tree of this method:

Solve $T(n) = 2T(n/2) + dn$, where $d > 0$ is constant.



$a^{\log_b n} = 2^{\log_2 n} = n^{\log_2 2} = n$

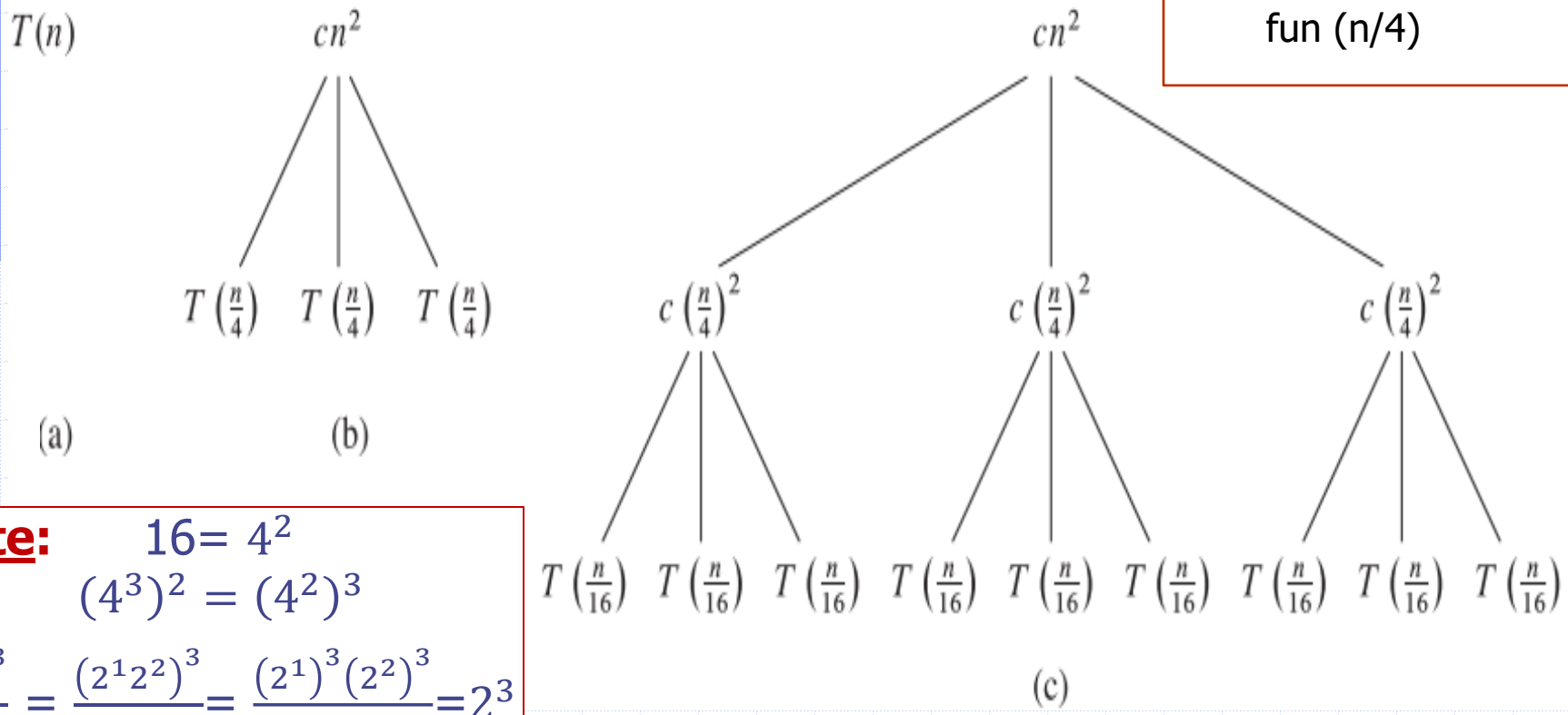
Total $\Theta(n \log n)$

$a^{\log_b x} = x^{\log_b a}$

The recursion-tree method

$$T(n) = 3T(n/4) + \theta(n^2)$$

```
Algorithm fun(n)
    if n>1
        for (n)
            for(n)
                print()
        fun (n/4)
        fun (n/4)
        fun (n/4)
```

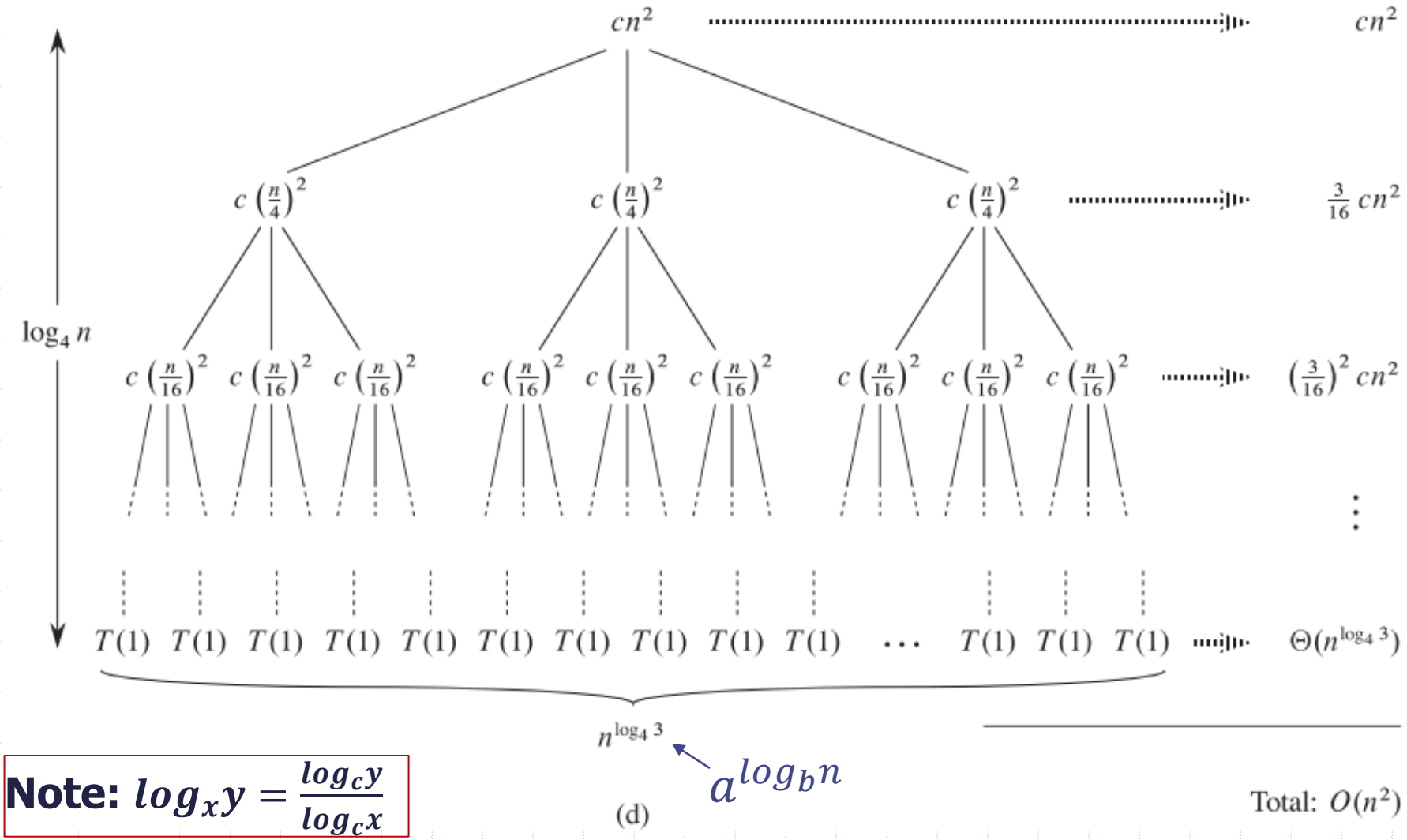


Note: $16 = 4^2$

$$(4^3)^2 = (4^2)^3$$

$$\frac{(2^3)^3}{(2^2)^3} = \frac{(2^1 2^2)^3}{(2^2)^3} = \frac{(2^1)^3 (2^2)^3}{(2^2)^3} = 2^3$$

The recursion-tree method



Note: $log_x y = \frac{log_c y}{log_c x}$

The recursion-tree method

- Now add up the costs over all levels to determine the cost for the entire tree :

$$T(n) = cn^2 + \frac{3}{16}cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \ldots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 + \Theta(n^{\log_4 3})$$

$$= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta\left(n^{\log_4 3}\right)$$

$$< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta\left(n^{\log_4 3}\right)$$

$$= \frac{1}{1-(3/16)} cn^2 + \Theta\left(n^{\log_4 3}\right)$$

$$= O\left(n^2\right).$$

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}.$$

The recursion-tree method

- T(n) **(Part a)** is expanded into equivalent tree **(Part b).** **cn^2** term (at the root) represents the cost at the top level of recursion, and the three subtrees of the root represent the cost incurred by the subproblems of size **n/4**.
- **Part c** expands each node of cost T(n/4) (in **part b)**. The cost for each of the three children of the root is $c(n/4)^2$.
- Since the subproblem sizes decrease as we get further from the root, we will reach a boundary condition.

The recursion-tree method

- $T(n) = 3T(n/4) + \theta(n^2)$
- How far from the root do we reach one?
  - The subproblem size for a node at depth ***i*** is $\mathbf{n/4^i}$. $\mathbf{=n/b^i}$
  - Thus, the subproblem size hits **n=1** when $\mathbf{n/4^i=1}$. i.e., $\mathbf{i=\log_4 n}$.
  - Thus the tree has $\mathbf{\log_4 n + 1}$ levels (at depths 0, 1, 2, …, $\log_4 n$).

The recursion-tree method

- To determine the cost at each level of the tree:
  - Each level has three times more nodes than the level above, so the number of nodes at depth **i** is $\mathbf{3^i}$.
  - Each node at depth **i** has a cost of $\mathbf{c(n/4^i)^2}$, for i=0, 1, 2, ….., $\log_4 n$ -1. (level before last one)
  - Multiplying, we get that total cost over all nodes at depth **i** is $\mathbf{3^i}$ $\mathbf{c(n/4^i)^2 = (3/16)^i\, cn^2}$.

$\mathbf{=a^i * f(n/bi)}$

- The last level at depth $\mathbf{\log_4 n}$:
  - It has $\mathbf{3^{\log_4 n} = n^{\log_4 3}}$ nodes, each contributing cost **T(1).**
  - A total cost of $\mathbf{n^{\log_4 3}}$ **.T(1)** is $\mathbf{\Theta(n^{\log_4 3})}$.

The Master Method

The master method

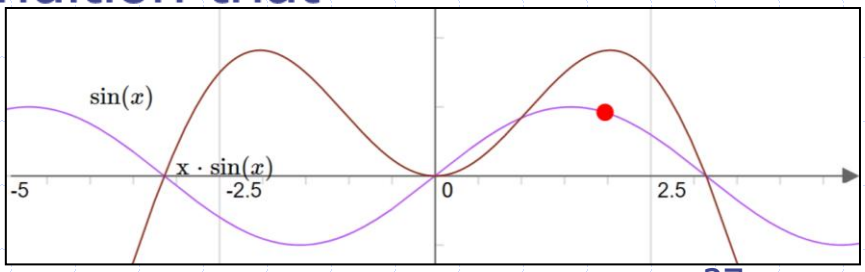
- The master method provides a "cookbook" method for solving recurrences of the form:

$$T(n) = a\,T(n/b) + f(n) \text{ where } a \geq 1 \text{ and } b > 1$$

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$. ■

The master method

- Compare the function f (n) with the function $n^{\log_b a}$
- Intuitively, the larger of the two functions determines the solution to the recurrence.
- In the first case, not only must f(n) be smaller than $n^{\log_b a}$, it must be polynomially smaller.
- In the third case, not only must f (n) be larger $n^{\log_b a}$, it also must be polynomially larger and in addition satisfy the "regularity" condition that a f(n/b) <= c f(n).



The master method

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$. ■

Example 1: $T(n) = 9\,T\left(\frac{n}{3}\right) + n$

$a = 9$

$b = 3$

$f(n) = n$

$n^{log_b a} = n^{log_3 9} = \theta(n^2)$?f (n) = $O(n^{2-1})$

Since $f(n) = O(n^{log_3 9 - \epsilon})$ for $\epsilon = 1$, case 1 applies

$T(n) = n^{log_b a} = n^{log_3 9} = \boldsymbol{\theta(n^2)}$

The master method

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$. ■

Example 2: $T(n) = T\left(\frac{2n}{3}\right) + 1$

$a = 1$

$b = 3/2$

$f(n) = 1$

$n^{log_b a} = n^0 = 1$

Since $f(n) = \theta(n^{log_b a})$, case 2 applies

$T(n) = \theta(n^{log_b a} \log n) = \theta(\log n)$

The master method

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$. ■

Example 3: $T(n) = 3T\left(\frac{n}{4}\right) + n\,logn$

$a = 3$

$b = 4$

$f(n) = n\,logn = Omega\,(n^{0.793+\,0.2}) = Omega\,(n^{1})$

$n^{log_b a} = n^{log_4 3} = O(n^{0.793})$

Since for $\epsilon \approx 0.2$, $f(n) = \Omega\left(n^{log_4 3 + \epsilon}\right)$, case 3 applies if $af\left(\frac{n}{b}\right) \leq cf(n)$ for some $c < 1$ and all sufficiently large n.

$3\frac{n}{4}\log\frac{n}{4} \leq cn\,logn$ for $c = 3/4$

$T(n) = \theta(n\,logn)$

The master method

- Example 4 :

  T(n) = 2 T (n/2) + n logn

  a = 2 , b =2 , f(n) = n log n

  $n^{\log_b a}$ = n ➔ f(n) > n

  But it is not polynomially larger !!

  n log n ? $n^{1+\epsilon}$ ➔ n log n ? n . n^{ϵ}

  Log n < n^{ϵ} for any $\epsilon > 0$

NOTE: It is not *polynomially* larger and is asymptotically less than n . The recurrence falls into the gap between case 2 and case 3.

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \le cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$. ■

T(n)	Case	O(?)
$T(n)= 9\,T(n/3) + 1$		
$T(n)= 8\,T(n/2) + n \log n$		
$T(n)= 8\,T(n/2) + n^2$		
$T(n)= 2\,T(n/2) + n$		
$T(n)= 4\,T(n/2) + n^2$		
$T(n)= 8\,T(n/2) + n^3$		
$T(n)= \ T(n/2) + n^2$		
$T(n)= \ 2\,T(n/2) + n^2$		
$T(n)= \ 2\,T(n/2) + N^2 \log^2 n$		

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \le cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$. ■

T(n)	Case	O(?)
T(n)= 9 T(n/3) + 1	1	N^2
T(n)= 8 T(n/2) + n log n	1	N^3
T(n)= 8 T(n/2) + n^2	1	n^3
T(n)= 2 T(n/2) + n	2	N log n
T(n)= 4 T(n/2) + n^2	2	N^2 log n
T(n)= 8 T(n/2) + n^3	2	N^3 log n
T(n)= T(n/2) + n^2	3	N^2
T(n)= 2 T(n/2) + n^2	3	N^2
T(n)= 2 T(n/2) + N^2 $\log^2 n$	3	N^2 $\log^2 n$